Lecture 13 — Priority Queues, Ordered Sets/Maps, Policy-Based Data Structures

> Year Curriculum · Prerequisites: Week 2 (STL basics), Week 4 (sorting & greedy), Week 5 (binary search), Week 12 (stacks & queues) · Estimated time: 2 × 90 min

1. Overview

Sorting (Week 4) and binary search (Week 5) are perfect when the data is fixed. Many problems, however, interleave updates and queries: tickets are sold, lights are installed, rooms are freed. Re-sorting after each change costs O(n log n); inserting into a sorted `vector` costs O(n). This lecture covers the standard library structures that keep data organised **while it changes**, with O(log n) per operation:

- `priority_queue`, a binary heap giving fast access to the maximum (or minimum);
- `set`, `multiset`, `map`, balanced binary search trees that keep keys sorted and answer "nearest key ≥ x" queries;
- `unordered_map`, a hash table, and when (not) to use it;
- the GNU policy-based `tree`, an ordered set that also answers "k-th smallest" and "how many are < x".

You will use these structures constantly: heaps in greedy algorithms and Dijkstra (Week 17), sets in sweep-line problems (Regionals), maps for coordinate compression and counting. Segment and Fenwick trees (Weeks 23–24) later solve the range-query problems that these structures cannot.

2. Priority queues

2.1 The binary heap

A binary heap stores n elements in an array that represents a complete binary tree: the children of index i are 2i + 1 and 2i + 2, and its parent is (i − 1)/2. The **heap property** says every parent is ≥ its children, so the maximum is at the root.

- **push**: append the element at the end and *sift up*, swapping with the parent while it is larger. The tree has height $\lfloor \log_2 n \rfloor$, so this is O(log n).
- **pop** (remove the maximum): move the last element to the root and *sift down*, swapping with the larger child while a child is larger. O(log n).

- **top**: read the root, O(1).
- Building a heap from n elements at once is O(n) (the `priority_queue` constructor from a range does this).

A heap is *not* sorted: you can see only the top. That is the trade-off — it is simpler and about 2–3 times faster than a balanced tree, but supports fewer operations.

2.2 `std::priority_queue`

```
priority_queue<int> pq;                                    // max-heap
priority_queue<int, vector<int>, greater<int>> minpq;      // min-heap
priority_queue<pair<long long, int>,
               vector<pair<long long, int>>, greater<>> dj;   // min by first, then
second
```

Operations: `push`, `top`, `pop` (returns `void`), `size`, `empty`. There is no iteration, no lookup, no erase of an arbitrary element, and no decrease-key.

The comparator is the source of endless confusion. `priority_queue<T, vector<T>, Cmp>` keeps at the top the element that is **largest** according to `Cmp`, where `Cmp(a, b) == true` means "a comes before b in sorted order", i.e. "a has lower priority". With the default `less<T>` the top is the maximum; with `greater<T>` it is the minimum. For a custom struct, a lambda `[](const Job& a, const Job& b) { return a.deadline > b.deadline; }` gives a min-heap on deadline. The negation trick (push `-x` into a max-heap) also yields a min-heap, at the cost of remembering every minus sign.

2.3 Patterns

- **Always process the best available item.** Merge k sorted lists; repeatedly combine the two smallest elements (Huffman coding, "minimum total cost of merging ropes"); simulate events in time order.
- **Resource assignment.** Keep a min-heap of "time when each machine/room becomes free". Worked example — CSES 1164 *Room Allocation*: sort customers by arrival; for each customer [a, b], if the room with the earliest departure is free before day a (departure < a), reuse it, otherwise open a new room; push (b, room). For (1,2), (2,4), (4,4): the first gets room 1; the second arrives on day 2 while room 1 is occupied until day 2 inclusive, so it opens room 2; the third arrives on day 4, room 1 is free (2 < 4), so it reuses room 1. Answer: 2 rooms, assignment `1 2 1`. Why is this greedy optimal? The number of rooms opened equals the maximum number of customers present on the same day at some moment, which is a lower bound for any assignment.
- **Top-k of a stream.** Keep a min-heap of size k; if a new element beats the top, pop and push. O(n log k).
- **Regret greedy.** Take items greedily; if a constraint is violated, undo the worst choice so far (pop the heap). Codeforces 1526C2 *Potions* drinks every potion and, whenever health goes

negative, un-drinks the most harmful one so far.
- **Lazy deletion.** When an element's priority changes, push the new version and leave the old one in the heap; when an outdated entry reaches the top, discard it. Dijkstra (Week 17) uses exactly this.

3. Ordered sets and maps

3.1 `set` and `multiset`

`set<T>` is a balanced binary search tree (a red-black tree in GCC) holding unique keys in sorted order. `insert`, `erase`, `find` and `count` are O(log n), `begin()` is the minimum, `rbegin()` the maximum, and range-for iterates in sorted order. `multiset<T>` is the same but allows duplicates.

3.2 Bound queries

The most useful operations are the member functions

- `s.lower_bound(x)`: iterator to the first element **≥ x**;
- `s.upper_bound(x)`: iterator to the first element **> x**.

Both return `s.end()` if no such element exists. From them, the other "nearest" queries follow by stepping back one position:

Query	Code
smallest element ≥ x	`auto it = s.lower_bound(x); if (it != s.end()) ...`
smallest element > x	`auto it = s.upper_bound(x); ...`
largest element ≤ x	`auto it = s.upper_bound(x); if (it != s.begin()) { --it; ... }`
largest element < x	`auto it = s.lower_bound(x); if (it != s.begin()) { --it; ... }`

Two traps. First, you must check against `end()` before dereferencing, and against `begin()` before decrementing. Second, the free function `std::lower_bound(s.begin(), s.end(), x)` *compiles* on a set but runs in O(n), because set iterators are not random-access. Always call the member `s.lower_bound(x)`.

3.3 Erasing from a multiset

`ms.erase(x)` with a **value** removes *every* copy of x. To remove a single copy, erase by **iterator**: `ms.erase(ms.find(x))` — and make sure `find` did not return `end()`, because erasing `end()` is undefined behaviour (usually a crash). This is probably the single most common STL bug in contests.

3.4 Worked example: CSES 1091 Concert Tickets

Each customer with budget t buys the most expensive remaining ticket with price ≤ t. Store the prices in a `multiset`. For budget t, `upper_bound(t)` gives the first price > t; if that is `begin()`, no ticket is affordable; otherwise step back one position to get the largest price ≤ t, print it, and erase that iterator. With prices `5 3 7 8 5` and budgets `4 8 3`, the answers are 3, 8 and −1. Total O((n + m) log n).

3.5 Worked example: CSES 1163 Traffic Lights

A street [0, x] receives lights one at a time; after each, print the length of the longest unlit segment. Keep two structures: `set<int> pos` with the light positions plus the endpoints 0 and x, and `multiset<int> len` with the lengths of the current segments. Adding a light at p splits the segment (a, b) that contains it, where b = `*pos.upper_bound(p)` and a is the element just before it. Erase **one** copy of b − a from `len`, insert p − a and b − p, and report `*len.rbegin()`. For x = 8 and lights 3, 6, 2 the answers are 5, 3, 3. (An elegant offline alternative processes the lights in reverse, *removing* them, where the maximum only grows.)

3.6 `map` and `unordered_map`

`map<K, V>` is a sorted dictionary on the same tree. `m[k]` returns a reference to the value for k, **inserting a default-constructed value if k is absent**. That makes counting easy (`freq[w]++`) but means a lookup like `if (m[k] > 0)` adds k to the map — it changes `size()` and can invalidate reasoning about which keys exist. Use `m.count(k)` or `m.find(k)` to test membership. `lower_bound` and `upper_bound` work on keys, so a map can answer "the entry with the largest key ≤ x", e.g. in interval bookkeeping.

`unordered_map<K, V>` is a hash table with O(1) *average* operations, but no ordering and an O(n) worst case. On Codeforces, open hacking means someone can craft keys that all collide in the default `std::hash`, turning your solution quadratic. If you use `unordered_map` there, add a randomized custom hash (e.g. splitmix64 with a time-based seed). In most problems the difference between O(log n) and O(1) does not matter, and `map` is the safer default. If keys are small integers (≤ 10^7), a plain array beats both; if keys are large but known in advance, sort and compress them to 0..n−1 (coordinate compression) and use an array.

4. Policy-based data structures

GCC ships a non-standard library, `__gnu_pbds`, whose `tree` container is a red-black tree augmented with subtree sizes. Declared as below, it behaves like a `set` with two extra O(log n) operations:

- `find_by_order(k)`: iterator to the k-th smallest element (0-indexed), or `end()` if k ≥ size;

- `order_of_key(x)`: the number of elements strictly less than x.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <class T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
                         tree_order_statistics_node_update>;
```

It works on Codeforces, AtCoder, CSES and every judge using GCC; it does not exist with MSVC or (usually) Clang with libc++.

Duplicates. Like `set`, the tree ignores duplicate inserts. To get an ordered multiset, insert pairs (value, unique id). Then `order_of_key({x, INT_MIN})` counts elements < x, `order_of_key({x, INT_MAX})` counts elements ≤ x, and `erase(lower_bound({x, INT_MIN}))` removes one copy of x. You will see code using `less_equal<T>` as the comparator to allow duplicates; it breaks `find`, `erase` and `lower_bound` semantics in subtle ways, so avoid it.

Worked example — CSES 2162 Josephus Problem II. Children 1..n stand in a circle; starting from the first, skip k children and remove the next, repeatedly. Keep the remaining children in an ordered set and the current index `pos`. Each round, `pos = (pos + k) % size`, remove `find_by_order(pos)`, and do not advance `pos`, because the next child slides into that index. For n = 7, k = 2 the removal order is 3 6 2 7 5 1 4. Each round is O(log n), so the total is O(n log n) instead of O(nk) with a linked list.

Other classic uses: counting inversions online (for each new element, count previous elements greater than it), sliding-window median (CSES 1076: k-th element of the window), and "how many values in the current set lie in [a, b]" (`order_of_key(b + 1) - order_of_key(a)`). When queries are offline, a Fenwick tree over compressed coordinates (Week 24) does the same job faster.

5. Choosing a structure

Ask which operations you need, then take the simplest structure that supports all of them:

Required operations	Structure
insert, read/remove the max (or min) only	`priority_queue`
insert, remove arbitrary, min and max	`multiset`
nearest key ≥ x / ≤ x under updates	`set` / `multiset` with member bounds
associate data with keys, ordered	`map`
associate data with keys, order irrelevant, speed critical	`unordered_map` (custom hash) or array after compression
k-th smallest / rank under updates	PBDS `ordered_set`

Required operations	Structure
no updates at all	sorted `vector` + `lower_bound` (fastest)

Constant factors are real: tree-based containers allocate a node per element and chase pointers, so they are 5–10 times slower than arrays. 10^6 operations take roughly 0.1–0.3 s, which is fine; 10^7 may not be.

6. Reference implementations

6.1 CSES 1091 — Concert Tickets

```cpp
#include <bits/stdc++.h>
using namespace std;
// CSES 1091 - Concert Tickets: each customer takes the most expensive ticket
// with price <= their max; print its price or -1. Tickets are removed once sold.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m; cin >> n >> m;
    multiset<int> tickets;
    for (int i = 0; i < n; i++) { int h; cin >> h; tickets.insert(h); }
    for (int i = 0; i < m; i++) {
        int t; cin >> t;
        auto it = tickets.upper_bound(t);       // first price > t
        if (it == tickets.begin()) { cout << -1 << '\n'; continue; }
        --it;                                   // largest price <= t
        cout << *it << '\n';
        tickets.erase(it);                      // erase ONE copy (by iterator)
    }
}
```

6.2 CSES 1164 — Room Allocation

```cpp
#include <bits/stdc++.h>
using namespace std;
// CSES 1164 - Room Allocation: assign rooms to n customers [a, b] (inclusive days),
// minimise the number of rooms; print k and a room for each customer.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<array<int, 3>> c(n);                         // {arrive, depart, id}
    for (int i = 0; i < n; i++) { cin >> c[i][0] >> c[i][1]; c[i][2] = i; }
    sort(c.begin(), c.end());
    // min-heap of {departure day, room number} for occupied rooms
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
    vector<int> room(n);
    int rooms = 0;
    for (auto [a, b, id] : c) {
        if (!pq.empty() && pq.top().first < a) {       // earliest-free room is free
```

```
            int r = pq.top().second; pq.pop();
            room[id] = r;
        } else {
            room[id] = ++rooms;
        }
        pq.push({b, room[id]});
    }
    cout << rooms << '\n';
    for (int i = 0; i < n; i++) cout << room[i] << " \n"[i == n - 1];
}
```

6.3 CSES 1163 — Traffic Lights

```
#include <bits/stdc++.h>
using namespace std;
// CSES 1163 - Traffic Lights: street [0, x]; after adding each light,
// print the length of the longest segment without lights.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int x, n; cin >> x >> n;
    set<int> pos = {0, x};          // light positions (plus both ends)
    multiset<int> len = {x};        // lengths of the current segments
    for (int i = 0; i < n; i++) {
        int p; cin >> p;
        auto right = pos.upper_bound(p);       // first position > p
        auto left = prev(right);               // last position < p
        int a = *left, b = *right;
        len.erase(len.find(b - a));            // ONE copy of the old segment
        len.insert(p - a);
        len.insert(b - p);
        pos.insert(p);
        cout << *len.rbegin() << " \n"[i == n - 1];
    }
}
```

6.4 CSES 2162 — Josephus Problem II (ordered set)

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
template <class T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
                         tree_order_statistics_node_update>;
// CSES 2162 - Josephus Problem II: children 1..n in a circle, every (k+1)-th
// child is removed (skip k). Print the removal order.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, k; cin >> n >> k;
    ordered_set<int> s;
```

```
    for (int i = 1; i <= n; i++) s.insert(i);
    int pos = 0;                                    // index (0-based) of current child
    while (!s.empty()) {
        pos = (pos + k) % (int)s.size();
        auto it = s.find_by_order(pos);             // k-th smallest, O(log n)
        cout << *it << " \n"[s.size() == 1];
        s.erase(it);                                // next child slides into index pos
    }
}
```

6.5 Ordered multiset and map demo

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
// Ordered multiset trick: store pairs {value, unique id}.
using omset = tree<pair<int, int>, null_type, less<pair<int, int>>, rb_tree_tag,
                   tree_order_statistics_node_update>;
int main() {
    omset s;
    int id = 0;
    for (int x : {5, 1, 5, 3, 9}) s.insert({x, id++});
    // number of elements strictly less than 5
    cout << "less than 5: " << s.order_of_key({5, INT_MIN}) << "\n";      // 2
    // number of elements <= 5
    cout << "at most 5:   " << s.order_of_key({5, INT_MAX}) << "\n";      // 4
    // median (0-indexed k = size/2)
    cout << "median:      " << s.find_by_order(s.size() / 2)->first << "\n"; // 5
    // erase one copy of 5
    s.erase(s.lower_bound({5, INT_MIN}));
    cout << "size after erasing one 5: " << s.size() << "\n";            // 4

    map<string, int> freq;
    for (string w : {"b", "a", "b", "c", "b"}) freq[w]++;
    for (auto& [w, c] : freq) cout << w << ':' << c << ' ';
    cout << "\n";
}
```

7. Common mistakes

- **Heap direction.** `priority_queue<int>` is a max-heap. For a min-heap use `greater<>`. With a custom comparator, remember it returns true when the first argument has *lower* priority.
- **`multiset::erase(value)`** removes all copies. Use `erase(find(value))` after checking `find(value) != end()`.
- **`std::lower_bound` on a set** is linear time. Use `s.lower_bound(x)`.
- **Dereferencing `end()`** after a failed bound query, or decrementing `begin()`.
- **`map::operator[]` in lookups** inserts default values; use `count` or `find`.

- **Modifying keys in place.** Set elements are `const`; to change one, erase it and insert the new value.
- **Iterator invalidation.** After `s.erase(it)`, `it` is invalid; use `it = s.erase(it)` when erasing while iterating. (Other iterators of a `set`/`map` stay valid.)
- **Hackable hashing** with `unordered_map` on Codeforces; use a custom hash or `map`.
- **Using a tree when an array suffices**: counting values ≤ 10^6 with `map` is 10× slower than `vector<int> cnt(1e6 + 1)`.

8. Practice problems

Warm-up:

1. **CSES 1621 — Distinct Numbers** — https://cses.fi/problemset/task/1621 (`set` size, or sort + unique).
2. **Codeforces 4C — Registration System** — https://codeforces.com/problemset/problem/4/C (`map<string, int>`).
3. **CSES 1640 — Sum of Two Values** — https://cses.fi/problemset/task/1640 (`map` from value to index).
4. **AtCoder ABC 141 D — Powerful Discount Tickets** — https://atcoder.jp/contests/abc141/tasks/abc141_d (max-heap, halve the top m times).

Core:

5. **CSES 1091 — Concert Tickets** — https://cses.fi/problemset/task/1091
6. **CSES 1164 — Room Allocation** — https://cses.fi/problemset/task/1164
7. **CSES 1163 — Traffic Lights** — https://cses.fi/problemset/task/1163
8. **AtCoder ABC 217 D — Cutting Woods** — https://atcoder.jp/contests/abc217/tasks/abc217_d (set + bounds).
9. **CSES 2162 — Josephus Problem II** — https://cses.fi/problemset/task/2162

Challenge:

10. **CSES 1076 — Sliding Window Median** — https://cses.fi/problemset/task/1076 (two multisets or an ordered set).
11. **Codeforces 1526C2 — Potions (Hard Version)** — https://codeforces.com/problemset/problem/1526/C2 (regret greedy with a min-heap).
12. **CSES 1144 — Salary Queries** — https://cses.fi/problemset/task/1144 (ordered multiset of pairs; later also solvable with a Fenwick tree).

9. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 4 "Data structures" (sets, maps, iterators, priority queues, policy-based sets): https://cses.fi/book/book.pdf

- cppreference: `std::priority_queue` https://en.cppreference.com/w/cpp/container/priority_queue, `std::set` https://en.cppreference.com/w/cpp/container/set, `std::multiset` https://en.cppreference.com/w/cpp/container/multiset
- Codeforces blog, "C++ STL: Policy based data structures" by adamant: https://codeforces.com/blog/entry/11080
- Codeforces blog, "Blowing up unordered_map, and how to stop getting hacked on it" by neal: https://codeforces.com/blog/entry/62393
- USACO Guide, Silver "More Operations on Sorted Sets": https://usaco.guide/silver/sorted-sets and Gold "Priority Queues" material.
- Coming up: Week 17 (Dijkstra with a heap), Weeks 23–24 (segment and Fenwick trees for range queries), Regionals "Advanced greedy" (exchange arguments, regret heaps).